The `Interface` class

As mentioned in the Quickstart, the `gr.Interface` class is a high-level abstraction in Gradio that allows you to quickly create a demo for any Python function simply by specifying the input types and the output types. Revisiting our first demo:

```
import gradio as gr

def greet(name, intensity):
    return "Hello, " + name + "!" * int(intensity)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "slider"],
    outputs=["text"],
    api_name="predict"
)

demo.launch()
```

We see that the `Interface` class is initialized with three required parameters:

- `fn`: the function to wrap a user interface (UI) around
- `inputs`: which Gradio component(s) to use for the input. The number of components should match the number of arguments in your function.
- `outputs`: which Gradio component(s) to use for the output. The number of components should match the number of return values from your function.

In this Guide, we'll dive into `gr.Interface` and the various ways it can be customized, but before we do that, let's get a better understanding of Gradio components.

Gradio Components

Gradio includes more than 30 pre-built components (as well as many **community-built *custom components***) that can be used as inputs or outputs in your demo. These components correspond to common data types in machine learning and data science, e.g. the `gr.Image` component is designed to handle input or output images, the `gr.Label` component displays classification labels and probabilities, the `gr.LinePlot` component displays line plots, and so on.

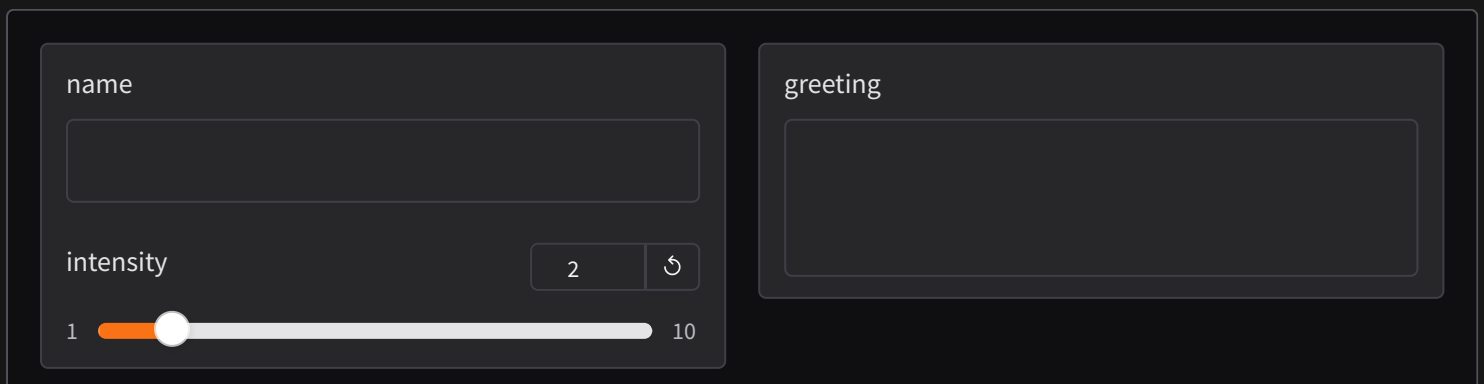
Components Attributes

We used the default versions of the `gr.Textbox` and `gr.Slider`, but what if you want to change how the UI components look or behave?

Let's say you want to customize the slider to have values from 1 to 10, with a default of 2. And you wanted to customize the output text field — you want it to be larger and have a label.

If you use the actual classes for `gr.Textbox` and `gr.Slider` instead of the string shortcuts, you have access to much more customizability through component attributes.

```
import gradio as gr

def greet(name, intensity):
    return "Hello, " + name + "!" * intensity

demo = gr.Interface(
    fn=greet,
    inputs=["text", gr.Slider(value=2, minimum=1, maximum=10, step=1)],
    outputs=[gr.Textbox(label="greeting", lines=3)],
    api_name="predict"
)

demo.launch()
```



Clear Submit Share via Link

gradio/hello_world_2 built with Gradio. Hosted on Spaces

Multiple Input and Output Components

Suppose you had a more complex function, with multiple outputs as well. In the example below, we define a function that takes a string, boolean, and number, and returns a string and number.

```
import gradio as gr

def greet(name, is_morning, temperature):
    salutation = "Good morning" if is_morning else "Good evening"
    greeting = f"{salutation} {name}. It is {temperature} degrees today"
    celsius = (temperature - 32) * 5 / 9
    return greeting, round(celsius, 2)

demo = gr.Interface(
    fn=greet,
    inputs=["text", "checkbox", gr.Slider(0, 100)],
    outputs=["text", "number"],
    api_name="predict"
)
demo.launch()
```

name

is_morning

temperature 0

0 100

output 0

output 1

0

Share via Link

Clear Submit

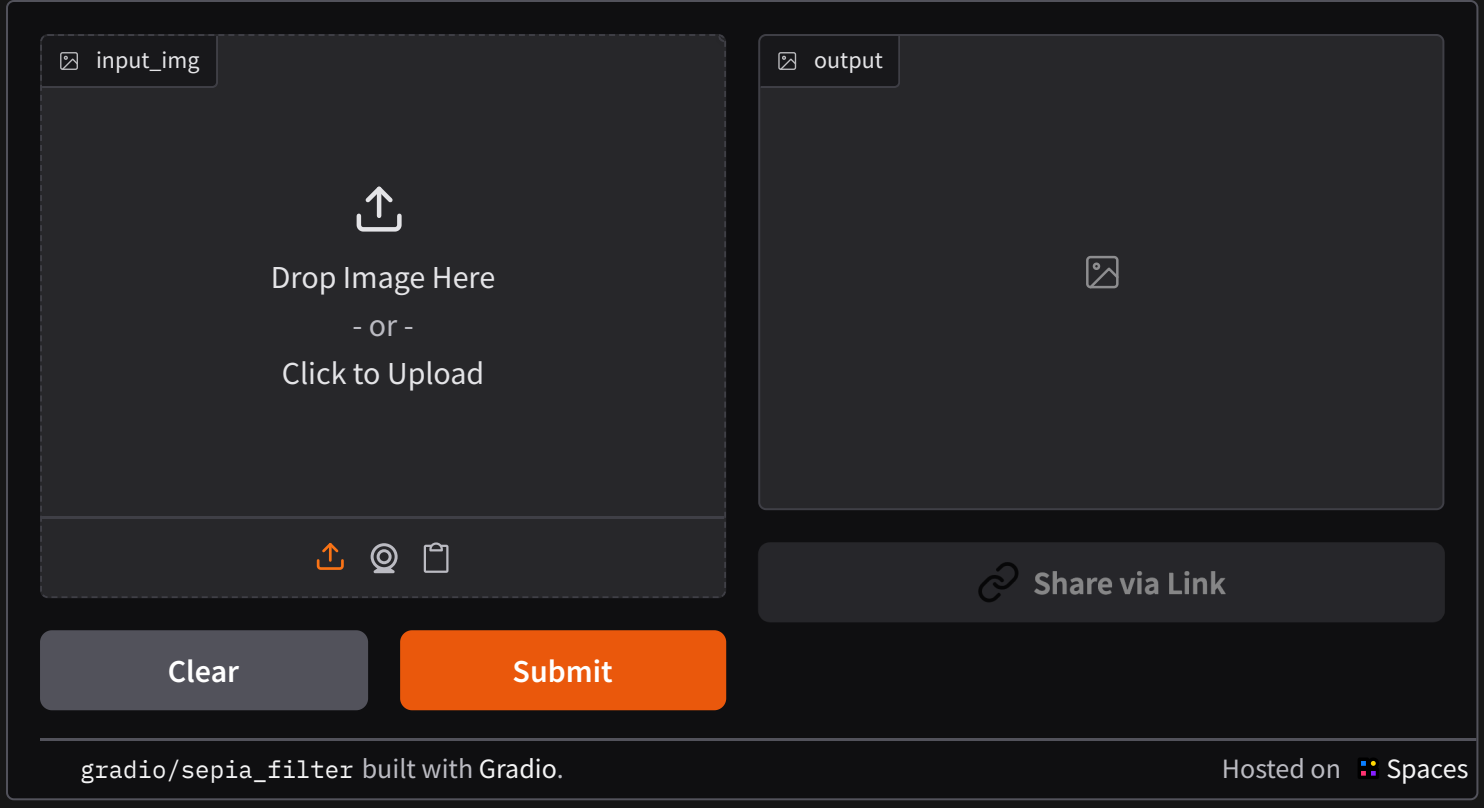
gradio/hello_world_3 built with Gradio. Hosted on Spaces

Just as each component in the `inputs` list corresponds to one of the parameters of the function, in order, each component in the `outputs` list corresponds to one of the values returned by the function, in order.

An Image Example

Gradio supports many types of components, such as `Image`, `DataFrame`, `Video`, or `Label`. Let's try an image-to-image function to get a feel for these!

```
import numpy as np
import gradio as gr

def sepia(input_img):
    sepia_filter = np.array([
        [0.393, 0.769, 0.189],
        [0.349, 0.686, 0.168],
        [0.272, 0.534, 0.131]
    ])
    sepia_img = input_img.dot(sepia_filter.T)
    sepia_img /= sepia_img.max()
    return sepia_img

demo = gr.Interface(sepia, gr.Image(), "image", api_name="predict")
demo.launch()
```



When using the `Image` component as input, your function will receive a NumPy array with the shape `(height, width, 3)`, where the last dimension represents the RGB values. We'll return an image as well in the form of a NumPy array.

Gradio handles the preprocessing and postprocessing to convert images to NumPy arrays and vice versa. You can also control the preprocessing performed with the `type=` keyword argument. For example, if you wanted your function to take a file path to an image instead of a NumPy array, the input `Image` component could be written as:

```
gr.Image(type="filepath")
```

You can read more about the built-in Gradio components and how to customize them in the Gradio docs.

Example Inputs

You can provide example data that a user can easily load into `Interface`. This can be helpful to demonstrate the types of inputs the model expects, as well as to provide a way to explore your dataset in conjunction with your model. To load example data, you can provide a **nested list** to the `examples=` keyword argument of the Interface constructor. Each sublist within the outer list represents a data sample, and each element within the sublist

represents an input for each input component. The format of example data for each component is specified in the Docs.

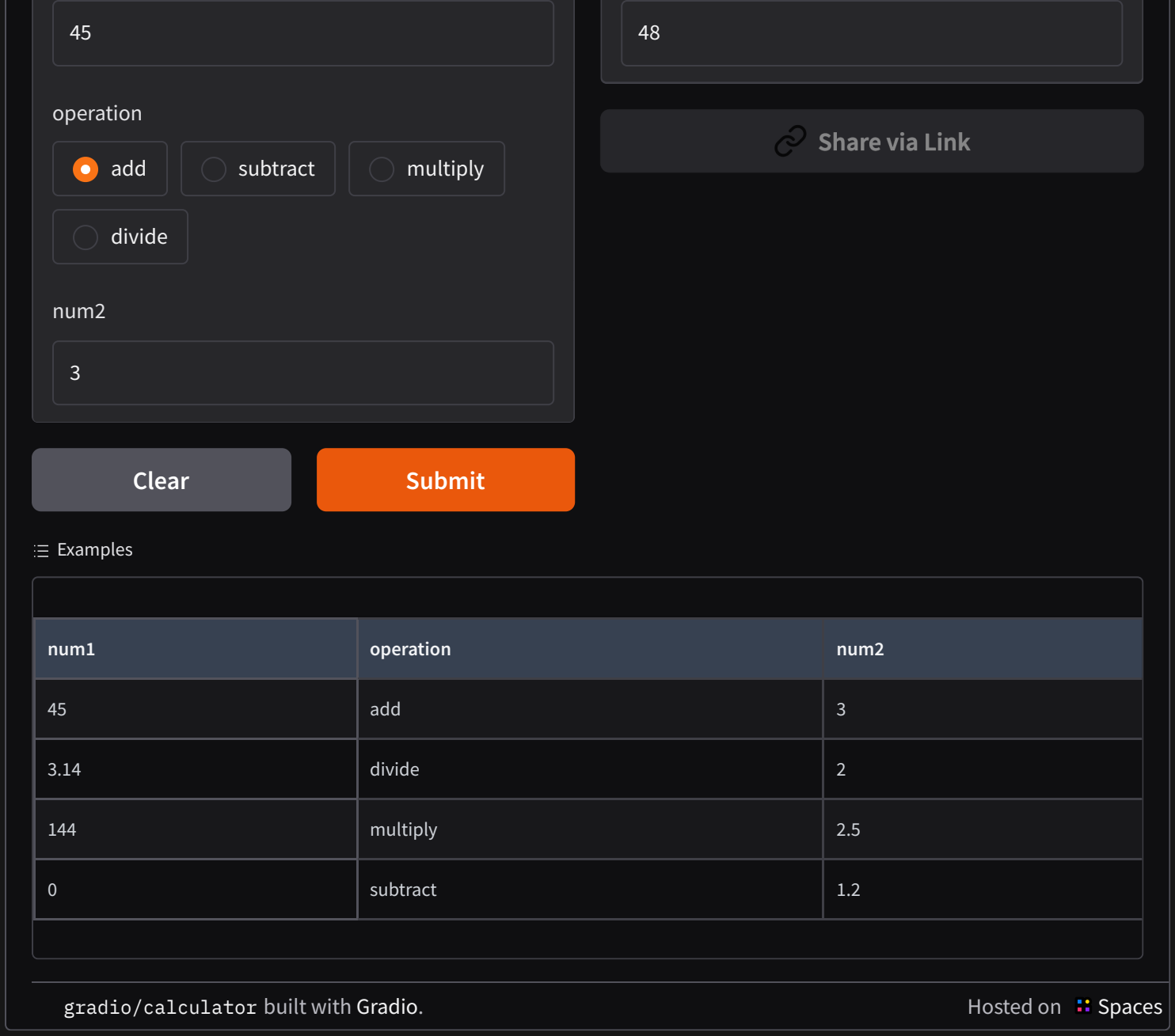
```
import gradio as gr

def calculator(num1, operation, num2):
    if operation == "add":
        return num1 + num2
    elif operation == "subtract":
        return num1 - num2
    elif operation == "multiply":
        return num1 * num2
    elif operation == "divide":
        if num2 == 0:
            raise gr.Error("Cannot divide by zero!")
        return num1 / num2

demo = gr.Interface(
    calculator,
    [
        "number",
        gr.Radio(["add", "subtract", "multiply", "divide"]),
        "number"
    ],
    "number",
    examples=[
        [45, "add", 3],
        [3.14, "divide", 2],
        [144, "multiply", 2.5],
        [0, "subtract", 1.2],
    ],
    title="Toy Calculator",
    description="Here's a sample toy calculator.",
    api_name="predict"
)

demo.launch()
```

Toy Calculator

Here's a sample toy calculator.

num1

output



You can load a large dataset into the examples to browse and interact with the dataset through Gradio. The examples will be automatically paginated (you can configure this through the `examples_per_page` argument of `Interface`).

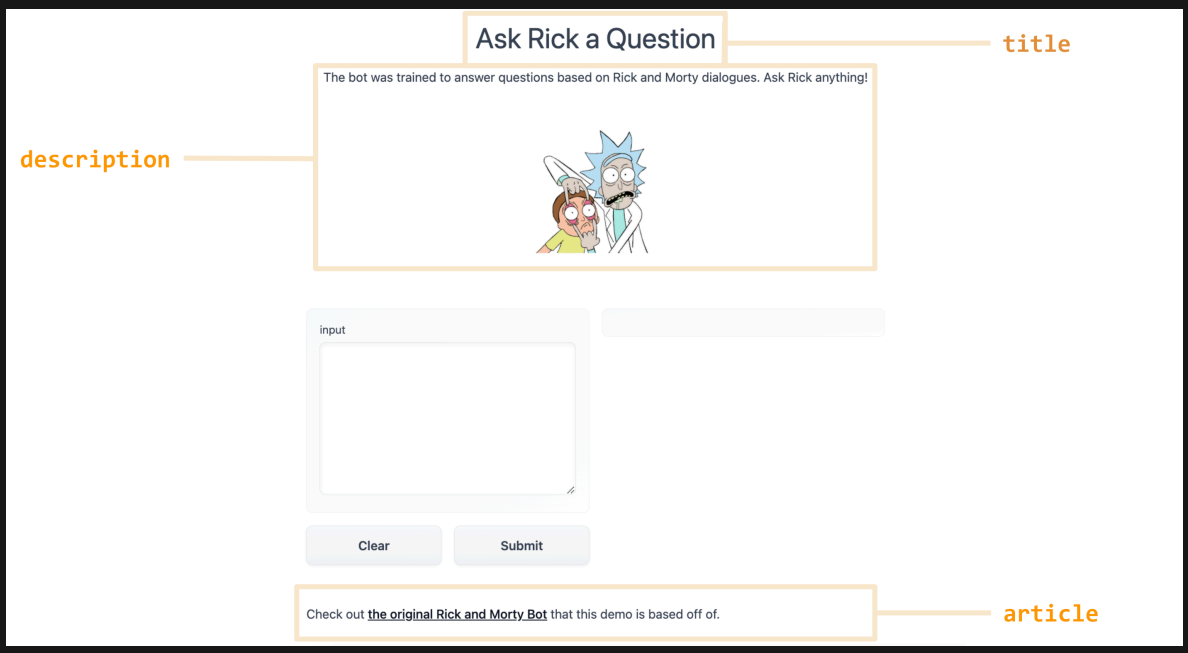
Continue learning about examples in the More On Examples guide.

Descriptive Content

In the previous example, you may have noticed the `title=` and `description=` keyword arguments in the `Interface` constructor that helps users understand your app.

There are three arguments in the `Interface` constructor to specify where this content should go:

- `title`: which accepts text and can display it at the very top of interface, and also becomes the page title.
- `description`: which accepts text, markdown or HTML and places it right under the title.
- `article`: which also accepts text, markdown or HTML and places it below the interface.



Another useful keyword argument is `label=`, which is present in every `Component`. This modifies the label text at the top of each `Component`. You can also add the `info=` keyword argument to form elements like `Textbox` or `Radio` to provide further information on their usage.

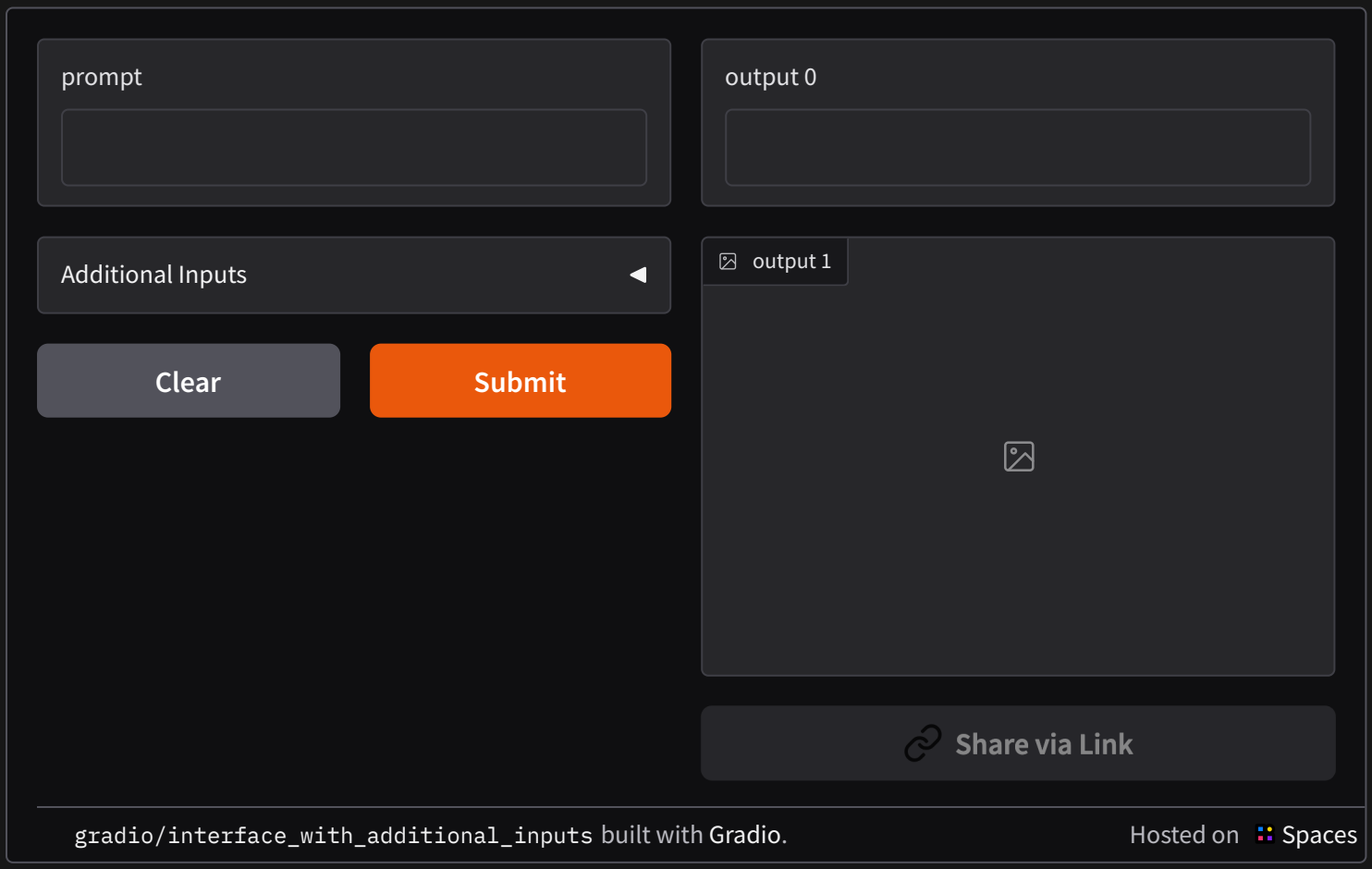
```
gr.Number(label='Age', info='In years, must be greater than 0')
```

Additional Inputs within an Accordion

If your prediction function takes many inputs, you may want to hide some of them within a collapsed accordion to avoid cluttering the UI. The `Interface` class takes an `additional_inputs` argument which is similar to `inputs` but any input components included here are not visible by default. The user must click on the accordion to show these components. The additional inputs are passed into the prediction function, in order, after the standard inputs.

You can customize the appearance of the accordion by using the optional `additional_inputs_accordion` argument, which accepts a string (in which case, it becomes the label of

the accordion), or an instance of the `gr.Accordion()` class (e.g. this lets you control whether the accordion is open or closed by default).

Here's an example:

```
import gradio as gr

def generate_fake_image(prompt, seed, initial_image=None):
    return f"Used seed: {seed}", "https://dummyimage.com/300/09f.png"

demo = gr.Interface(
    generate_fake_image,
    inputs=["textbox"],
    outputs=["textbox", "image"],
    additional_inputs=[
        gr.Slider(0, 1000),
        "image"
    ],
    api_name="predict",
)

demo.launch()
```